

本地场景短时活动规划 Agent

设计文档（Tool 实现 + Planning 策略 + 异常处理）

一、Planning 策略：Plan-then-Execute

核心思想：将传统多轮串行 ReAct 压缩为 3 个确定性阶段，减少 LLM 调用次数，提升响应速度。

阶段	说明
Plan	LLM 一次性输出完整工具调用计划（JSON 数组），包含每个工具的参数
Execute	按顺序逐个执行工具，收集 Observation 结果
Generate	LLM 基于所有工具结果，生成带时间轴的最终方案

Plan 阶段 System Prompt 关键约束：

- 先用 search_pois 搜索候选景点和餐厅（各 1 次）
- 对最合适餐厅调用 check_queue 确认排队
- 排队<30 分钟时调用 book_table 预订
- 工具总数控制在 3-5 个，避免过度调用

二、工具调用链路

4 个核心 Tool 的定义与调用顺序如下：

Tool	功能	调用时机
search_pois	搜索 POI（景点/餐厅），支持分类/预算/标签筛选	Plan 阶段最先调用
check_queue	查询餐厅实时排队和空位情况	search_pois 后对候选餐厅调用
book_table	预订餐厅桌位，返回确认码	排队可接受时调用
send_plan	将计划发送给指定收件人	Generate 后最终调用

典型调用链：search_pois(景点) → search_pois(餐厅) → check_queue → book_table → send_plan

每个工具返回统一结构：{success, data, error, _duration_ms}，便于链路中统一处理成败。

三、异常处理机制

3.1 三级降级策略

层级	触发条件	处理方式
L1 正常	LLM 可用 + Plan 解析成功	Plan-then-Execute 全流程
L2 降级	LLM 不可用 / Plan 解析失败	FallbackPlanner 规则模板生成方案
L3 兜底	Generate 阶段失败	从已获取 Observation 构造最小可行方案

3.2 意图层异常检测

- 极低预算：预算 Lv1 + 含“便宜/省钱”等词 → 提示优先推荐免费景点
- 跨城检测：含“少林寺/嵩山” → 提示距市区 80km，需预留 6-8 小时
- 时间紧张：可用时间<4 小时 → 建议精简至 2-3 个核心点

3.3 工具层容错

- 每个 Tool 用 try/except 包裹，异常时返回 success=False + error 信息
- 工具调用超时或失败不影响后续工具执行
- 结果解析失败（JSON 格式错误）自动重试，最多 2 次

四、Tool 实现代码（Mock API）

以下展示 search_pois、check_queue、book_table 三个核心 Tool 的完整实现。

4.1 search_pois

```
def tool_search_pois(params: dict, poi_data: list) -> dict:
    start = time.time()
    try:
        query = params.get("query", "")
        budget_level = params.get("budget_level", 2)
        prefer_tags = params.get("prefer_tags", [])
        category = params.get("category")
        district = params.get("district")
        time.sleep(0.05) # Mock API 延迟
        results = []
        for poi in poi_data:
            if category and poi.category != category:
                continue
            if district and poi.district != district:
                continue
            if budget_level == 1 and poi.price_avg > 50:
                continue
            elif budget_level == 2 and poi.price_avg > 180:
                continue
            results.append(poi.to_dict())
        results.sort(key=lambda p: p.get("rating", 0), reverse=True)
        results = results[:10]
        return {"success": True, "data": {"pois": results, "count": len(results)}, "error": None}
    except Exception as e:
        return {"success": False, "data": None, "error": f"POI 搜索异常: {str(e)}"}
```

4.2 check_queue

```
def tool_check_queue(params: dict) -> dict:
    start = time.time()
    try:
        restaurant_id = params.get("restaurant_id", "")
        party_size = params.get("party_size", 2)
        preferred_time = params.get("preferred_time", "18:00")
        time.sleep(0.08) # Mock API 延迟
        capacity = MOCK_RESTAURANT_CAPACITY.get(restaurant_id,
            {"total_tables": 30, "occupied": 20, "queue_length": 8})
        base_wait = capacity["queue_length"] * 5
        if party_size >= 6: base_wait += 15
        if preferred_time in ["18:00", "18:30", "19:00"]: base_wait += 10
        has_available = capacity["occupied"] < capacity["total_tables"] * 0.85
        wait_minutes = max(0, base_wait + random.randint(-5, 10))
        return {"success": True, "data": {
            "queue_length": capacity["queue_length"],
            "wait_minutes": wait_minutes,
            "has_available": has_available,
            "status": "可预订" if has_available and wait_minutes < 30 else "需排队",
        }, "error": None}
    except Exception as e:
        return {"success": False, "data": None, "error": f"排队查询异常: {str(e)}"}
```

4.3 book_table

```
def tool_book_table(params: dict) -> dict:
    start = time.time()
    try:
        restaurant_id = params.get("restaurant_id", "")
        restaurant_name = params.get("restaurant_name", "未知餐厅")
        party_size = params.get("party_size", 2)
        booking_time = params.get("time", "18:00")
        time.sleep(0.12) # Mock API 延迟
        booking_id = f"MT-{restaurant_id}-{int(time.time()) % 100000:05d}"
        return {"success": True, "data": {
            "booking_id": booking_id,
            "restaurant_name": restaurant_name,
            "party_size": party_size,
            "booking_time": booking_time,
            "status": "confirmed",
            "confirm_code": hashlib.md5(booking_id.encode()).hexdigest()[:6].upper(),
        }, "error": None}
    except Exception as e:
        return {"success": False, "data": None, "error": f"预订失败: {str(e)}"}
```

4.4 Tool 注册表

```
TOOL_REGISTRY = {
    "search_pois": {"name": "search_pois", "fn": tool_search_pois,
                    "params": ["query", "budget_level", "prefer_tags", "category", "district"]},
    "check_queue": {"name": "check_queue", "fn": tool_check_queue,
                    "params": ["restaurant_id", "party_size", "preferred_time"]},
    "book_table": {"name": "book_table", "fn": tool_book_table,
                   "params": ["restaurant_id", "party_size", "time"]},
    "send_plan": {"name": "send_plan", "fn": tool_send_plan,
                  "params": ["recipient", "plan_text", "channel"]},
}
```